Machine Learning Classification of Melanoma from fusing Skin Lesion Images and Demographic Data

Kate Li
BME Computing Final Project

Image

Patient Data

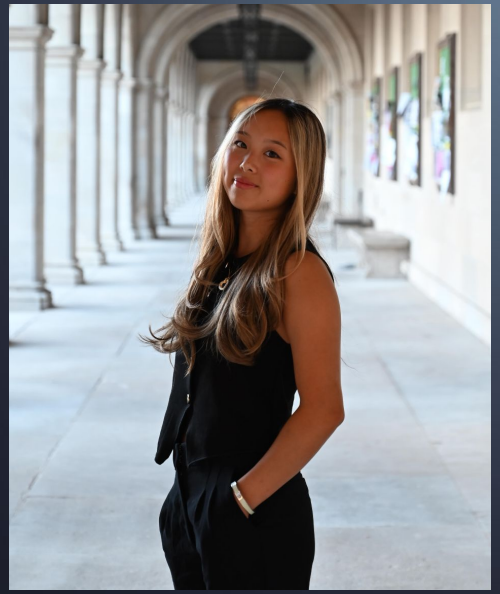
Age	Gender	Skin_Type	UV_Exposure_Level	Family_History	History_of_Sunburns	Mole_Count
69	Male	V	Low	0	1	13
32	Male	IV	High	0	4	11
89	Male	I	Moderate	0	3	15
78	Male	I	High	1	4	9
38	Female	V	Low	1	2	12

Prediction?

A little bit about me

Sophomore BME student
- Minors in Biomedical Data Science & Mechanical Engineering
- Premed...

Passionate about:
- Computer-aided diagnosis
- Combining them with physician conclusions to streamline/make decisions more accurate and confident



Kate Li

Project Rationale & Goals

Melanoma accounts for the majority of skin cancer **deaths**.

Over 100,000 new cases in U.S. each year with risk heavily linked to UV exposure and age

However, early detection has a 99% five-year survival rate, which drops the later diagnosed

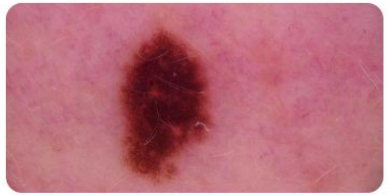
Age hinders ability to get in-person diagnoses & care

https://melanoma.org/facts-stats/

Data from Kaggle!

Melanoma Skin Cancer Dataset of 10000 Images

This dataset contains two classes of melanoma cancer, malignant and benign.



9600 images for training the model and 1000 images for evaluation of model.

Skin Cancer Prediction Using Clinical Data

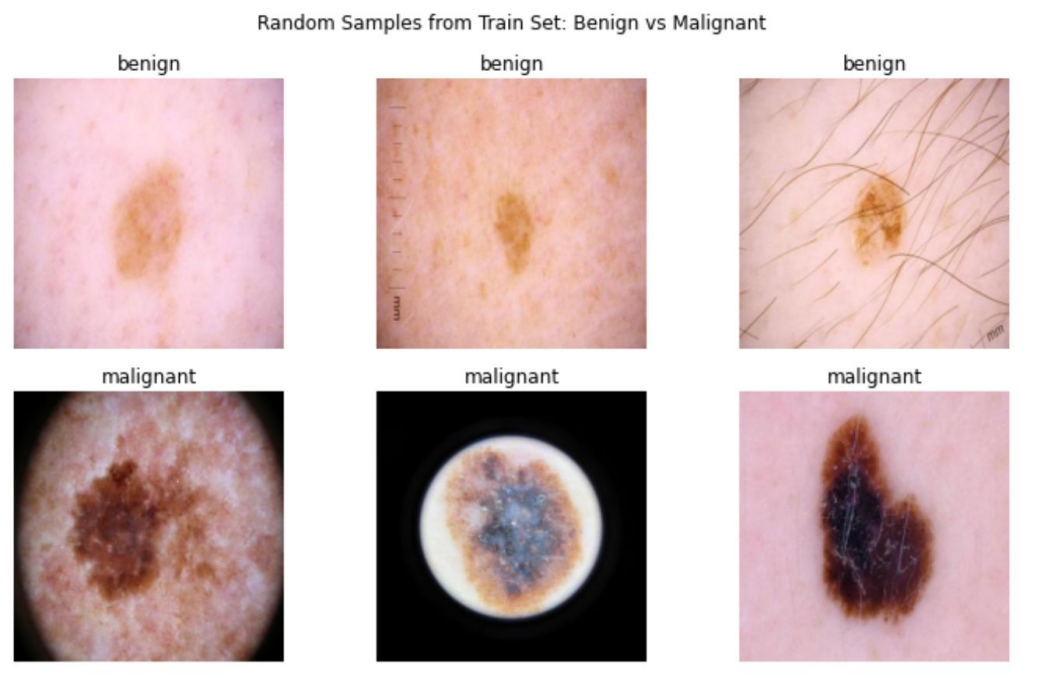
Clinical and lifestyle variables to support skin cancer risk analysis

3,000 synthetic patient records including: age, gender, skin type, UV exposure, and clinical markers (ex. mole count, family history).

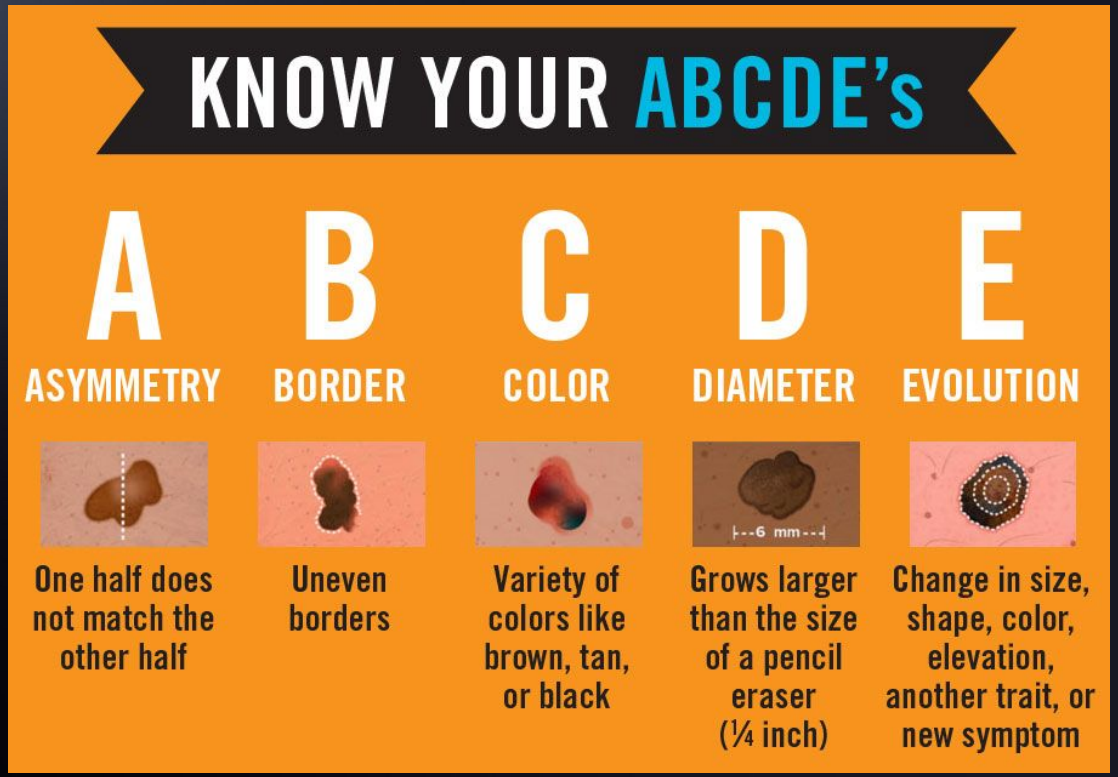
Binary target variable Cancer_Diagnosis: skin cancer (1) or not (0).

Data Visualization

```python
# Randomly select 3 images
random_images = random.sample(images, 3)

for img_name in random_images:
    img_path = os.path.join(category_path, img_name)
    img = Image.open(img_path)

    plt.subplot(2, 3, plot_index)
    plt.imshow(img)
    plt.title(category)
    plt.axis("off")

    plot_index += 1
```



Feature Extraction



https://dermatologyofct.com/what-we-do/skin-cancer-screening-treatment-2/melanoma/

04 Data Preprocessing

```
base_path = "melanoma_cancer_dataset"

splits = ["train", "test"]
categories = ["benign", "malignant"]
```

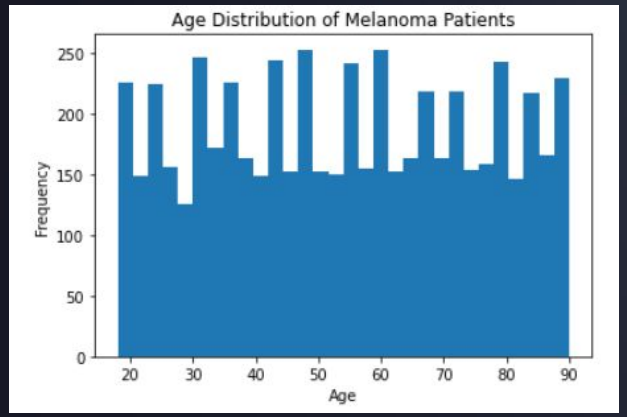
* train/test split not necessary because already allocates separate set of images for testing (no overfitting!)

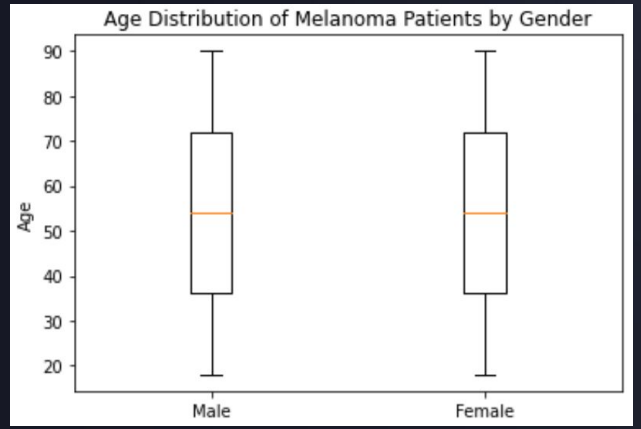
* throwing away images not needed, already clean dataset

```
TRAIN SET
--------------------
Benign: 5000 images
Malignant: 4605 images

TEST SET
--------------------
Benign: 500 images
Malignant: 500 images

after counting the images, I have determined that there is not a class imbalance, so no cleaning needs to be done
```

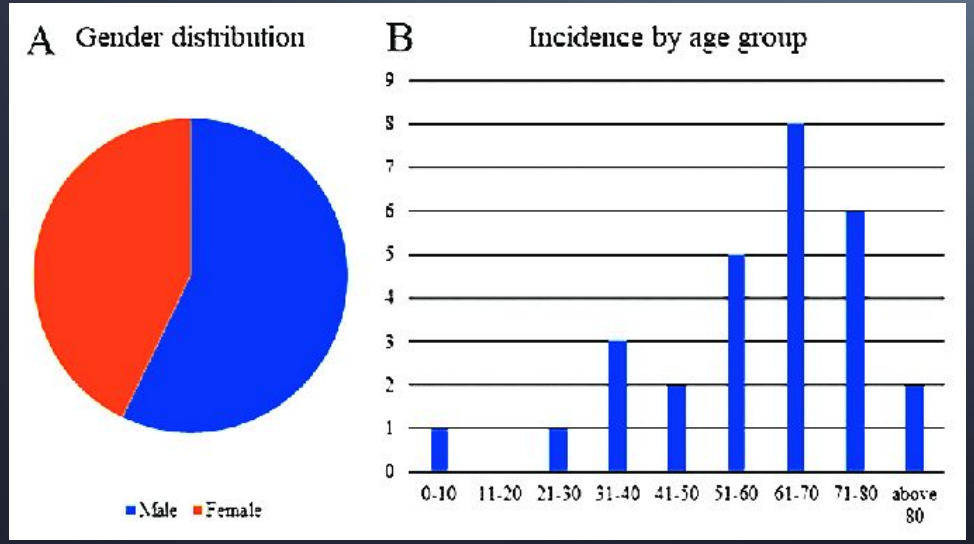
Fairly even split, no class imbalance!

Synthetic dataset problems





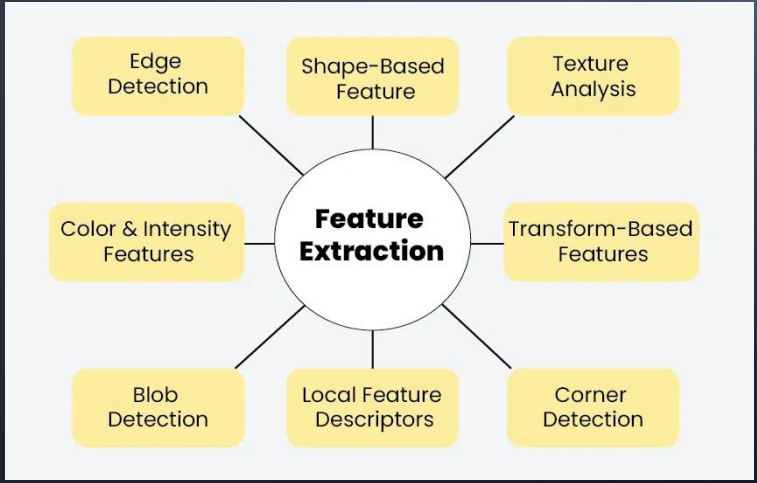
Actual:



https://www.researchgate.net/publication/352508004_Demographic_clinical_and_morphological_profile_of_malignant_cutaneous_melanoma_in_a_Northeastern_region_of_Bulgaria

Machine Learning: Binary Classification

- ➔ CNN (end-to-end) - automatically learn spatial features such as texture, asymmetry, and border irregularities
- ➔ Feature extractor (CNN) + Random Forest
- ➔ Feature extractor (CNN) + SVM



```python
# Build CNN model
model = keras.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(img_height, img_width, 3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')  # binary output
])
```

Relu to learn the non-linear patterns

Sigmoid to map the output to a probability between 0 and 1!

```
Epoch 1/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 171s 556ms/step - accuracy: 0.7177 - loss: 0.5453 - val_accuracy: 0.8820 - val_loss: 0.2819
Epoch 2/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 129s 427ms/step - accuracy: 0.8677 - loss: 0.3186 - val_accuracy: 0.8960 - val_loss: 0.2754
Epoch 3/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 136s 452ms/step - accuracy: 0.8841 - loss: 0.2895 - val_accuracy: 0.8900 - val_loss: 0.2906
Epoch 4/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 141s 448ms/step - accuracy: 0.8854 - loss: 0.2797 - val_accuracy: 0.8800 - val_loss: 0.2626
Epoch 5/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 137s 454ms/step - accuracy: 0.8803 - loss: 0.2763 - val_accuracy: 0.8900 - val_loss: 0.2654
Epoch 6/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 128s 425ms/step - accuracy: 0.8909 - loss: 0.2637 - val_accuracy: 0.8970 - val_loss: 0.2608
Epoch 7/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 131s 435ms/step - accuracy: 0.8967 - loss: 0.2446 - val_accuracy: 0.8930 - val_loss: 0.2484
Epoch 8/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 174s 537ms/step - accuracy: 0.9045 - loss: 0.2431 - val_accuracy: 0.8930 - val_loss: 0.2596
Epoch 9/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 180s 463ms/step - accuracy: 0.9076 - loss: 0.2295 - val_accuracy: 0.8930 - val_loss: 0.2469
Epoch 10/10
301/301 ━━━━━━━━━━━━━━━━━━━━ 150s 497ms/step - accuracy: 0.9091 - loss: 0.2298 - val_accuracy: 0.9000 - val_loss: 0.2364
32/32 ━━━━━━━━━━━━━━━━━━━━ 4s 131ms/step - accuracy: 0.9065 - loss: 0.2467

Test Accuracy: 0.9000
```

```python
# False Positive
if fp_idx is not None:
    plt.subplot(1,3,2)
    plt.imshow(images_list[fp_idx])
    plt.title(f"False Positive\nTrue: benign\nPred: malignant")
    plt.axis("off")

# False Negative
if fn_idx is not None:
    plt.subplot(1,3,3)
    plt.imshow(images_list[fn_idx])
    plt.title(f"False Negative\nTrue: malignant\nPred: benign")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Total samples: 1000
Correct predictions: 900
False Positives (benign → malignant): 39
False Negatives (malignant → benign): 61

Percentages:
False Positive Rate: 0.0390
False Negative Rate: 0.0610
```

Hybrid ML Approach – CNN+ Random Forest

CNN acts as a feature extractor (detects textures, edges, lesion patterns)
Random Forest acts as a classifier on top of those features (splitting logic isn't based on distance ↓)

```
# Random Forest: Load data (same structure, but no normalization layer needed here)
```

```
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    labels='inferred',
    label_mode='binary',
    image_size=(img_height, img_width),
    batch_size=batch_size,
    shuffle=False   # IMPORTANT for alignment
)

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    labels='inferred',
    label_mode='binary',
    image_size=(img_height, img_width),
    batch_size=batch_size,
    shuffle=False
)
```

```
RandomForestClassifier
RandomForestClassifier(n_jobs=-1, random_state=42)
```

```
y_pred = rf_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy (Random Forest): {accuracy:.4f}")
```

```
Test Accuracy (Random Forest): 0.9030
```

Hybrid ML Approach – CNN + SVM

*Main additional step is feature scaling, important for SVMs but not Random Forests. Because distance matters in kernel space!

What to expect

- ➔ Random Forest is usually more robust and faster to train
- ➔ SVM can become slow if your feature vectors are very large (which they are after CNN flattening….)

Principal Component Analysis (PCA): preprocessing dimensionality reduction technique

- transforms high-dimensional data into a smaller set of variables (“principal components”) while retaining as much original variation as possible

```python
# SVM. Scale features
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

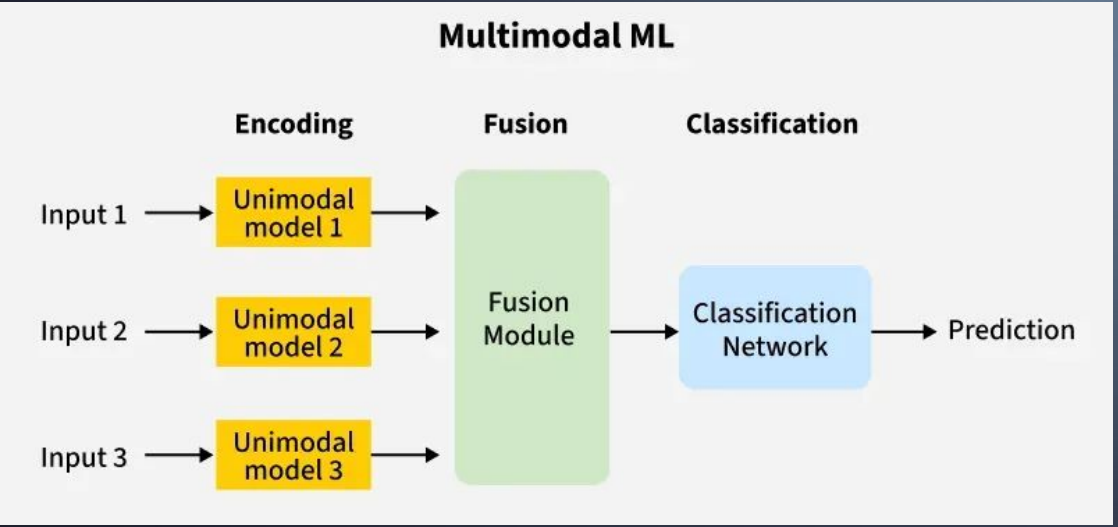
```python
# apply PCA for speedier
from sklearn.decomposition import PCA

pca = PCA(n_components=150, random_state=42)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Reduced shape:", X_train_pca.shape)
```

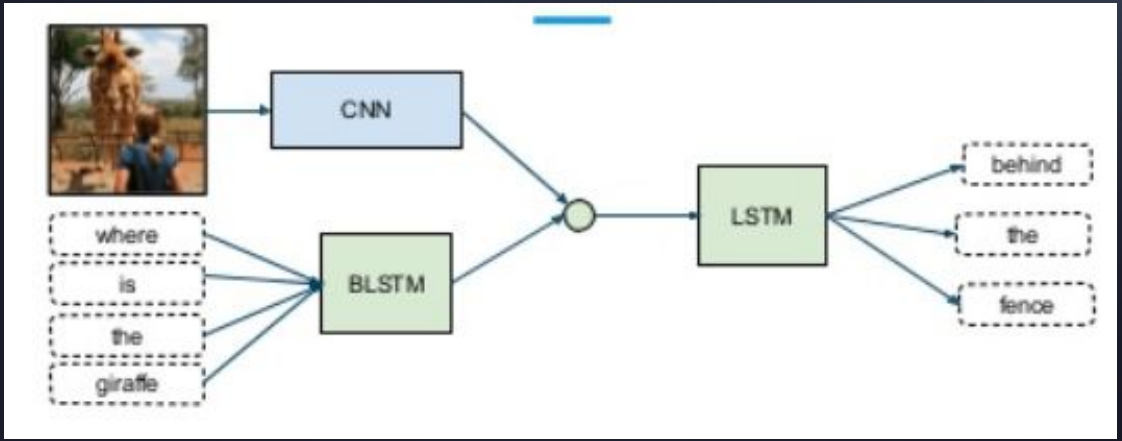
```python
# faster SVM (linear)
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

svm_model = LinearSVC(
    C=1.0,
    max_iter=5000
)

svm_model.fit(X_train_pca, y_train)

y_pred_svm = svm_model.predict(X_test_pca)

accuracy_svm = accuracy_score(y_test, y_pred_svm)
print(f"Test Accuracy (Fast SVM): {accuracy_svm:.4f}")
```

Comparison & Modifiers

- ➔ label_mode='binary' ensures labels are encoded as 0 (benign) and 1 (malignant), matching a two-class setup.
- ➔ ReLU (Rectified Linear Unit) is used in hidden layers to introduce nonlinearity while maintaining stable gradients and efficient computation.
- ➔ The final layer uses sigmoid, producing a probability between 0 and 1 for the malignant class.
- ➔ binary_crossentropy is the loss function for this binary probability output
- ➔ Adam optimizer for training because HIGH MOMENTUM (keeps track of past gradients) and RMSProp (adjusts learning rate)

Future Steps!





Questions?

Additional links if curious:

Multimodal Deep Learning: A Survey of Models, Fusion Strategies, Applications, and Research Challenges

Melanoma Skin Cancer Dataset of 10000 Images

Skin Cancer Prediction Using Clinical Data | Kaggle

CREDITS: This presentation template was created by **Slidesgo**, and includes icons, infographics & images by **Freepik**